

TDD (Technical Design Document) — ThesisGuard

ThesisGuard_Team_Guide.md의 아키텍처·DB 스키마·API·MCP Tool 설계를 기술 설계 문서 형식으로 재구성했습니다.

문서 정보

항목	내용
상태	Draft · v0.1
작성자 / 오퍼	ThesisGuard 팀 — B(Backend & Data Infra) 주도, C(AI/Agent Core) 공동
관련 문서	PRD · 기능정의서 · ADR
최종 수정	2026-07-10

1. 개요 & 범위

대상: ThesisGuard — 투자 논리 지속 검증 Agent 시스템.

다루는 것: 전체 아키텍처, DB 스키마, Vector Store, LangGraph 워크플로, REST API, MCP Tool, 배포 구성.

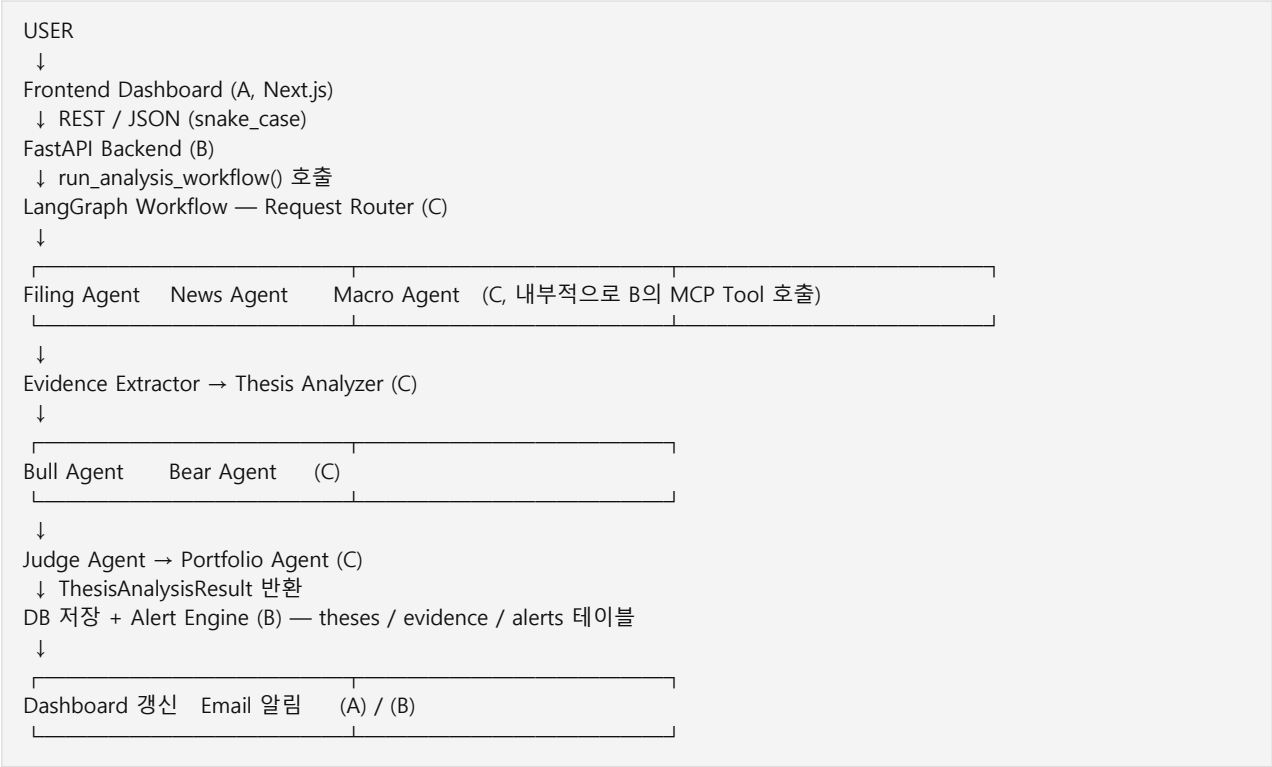
다루지 않는 것(설계 비목표): 자동매매 실행 엔진, 실시간(초단위) 시세 스트리밍 인프라, 다국어 처리 — PRD Non-Goals와 일치.

2. 설계 목표 & 제약

목표: ① 근거 기반 분석(할루시네이션 억제, 모든 판정에 citation) ② 3인이 겹치지 않고 병렬 개발 가능한 명확한 계약(API/함수 시그니처) ③ 확장 가능한 멀티 에이전트 파이프라인.

제약: 외부 LLM API 의존 · 외부 데이터 소스(SEC EDGAR, 시세 API, FRED) 의존 · 3인 소규모 팀 · 학기 PBL 일정.

3. 시스템 아키텍처



컴포넌트	책임
FastAPI Backend	인증·REST API·MCP Tool 서버·DB 접근 (B)
LangGraph Workflow	Request Router, 전체 분석 파이프라인 오케스트레이션 (C)
Filing/News/Macro Agent	MCP Tool을 LangChain Tool로 감싸 외부 데이터 수집 (C→B)
Evidence Extractor/Analyzer	수집 데이터를 근거 단위로 분리, 기존 Thesis와 대조 (C)
Bull/Bear/Judge Agent	Agentic Debate로 Thesis 변화 방향·강도 판정 (C)
Portfolio Agent	Thesis Concentration·Common Risk 분석 (C)
PostgreSQL	사용자/포트폴리오/Thesis/Evidence/Alert 등 정형 데이터 (B)
Vector DB (pgvector/Qdrant)	공시·IR·실적·뉴스 문서 임베딩 (B 적재, C 조회)
Alert Engine	severity 기반 즉시/주간 이메일 발송 (B)

4. 핵심 흐름 (Sequence — Thesis 분석 요청)

User → Frontend: "이 종목 분석해줘" 클릭
Frontend → Backend: POST /api/holdings/{id}/analyze
Backend → LangGraph: run_analysis_workflow(portfolio_id, holding_id)
LangGraph → Filing/News/Macro Agent: 병렬 데이터 수집 (MCP Tool 경유)
Filing/News/Macro Agent → Evidence Extractor: research_data
Evidence Extractor → Evidence Classifier: evidence_list
 alt 증거 불충분
 Evidence Classifier → LangGraph: Additional Research 루프 (재수집, 최대 N회)
 end
Evidence Classifier → Bull Agent / Bear Agent: 분류된 evidence_list
Bull Agent → Judge Agent: bull_report
Bear Agent → Judge Agent: bear_report
Judge Agent → LangGraph: updated_confidence, updated_status, change_reason
LangGraph → Backend: ThesisAnalysisResult
Backend → DB: theses / thesis_versions / evidence / analysis_results 저장
Backend → Alert Engine: severity 판정 결과 전달
Alert Engine → SMTP: (CRITICAL/MAJOR) 즉시 발송 / (MINOR) 주간 큐 적재
Backend → Frontend: 분석 결과 응답

5. 데이터 모델

모든 테이블은 B가 소유·마이그레이션(Alembic). PK는 UUID, **created_at/updated_at** 고정.

엔티티	주요 필드	비고
users	id, email(UNIQUE), password_hash, name	인증 주체
portfolios	id, user_id, name, investment_purpose, investment_horizon, cash_ratio	사용자별 복수 포트폴리오
holdings	id, portfolio_id, ticker, quantity, avg_buy_price, target_weight, current_weight	보유 종목
transactions	id, portfolio_id, type(BUY/SELL/REBALANCE/CASH_ADJUST), before_snapshot(JSONB), after_snapshot(JSONB)	리밸런싱 히스토리
theses	id, holding_id(UNIQUE), raw_input, main_thesis, key_assumptions(JSONB), positive_signals(JSONB), negative_signals(JSONB), key_risks(JSONB), confidence_score, status	1 holding : 1 thesis
thesis_versions	id, thesis_id, version_no, confidence_score, status, change_reason, conflicting_assumptions(JSONB), observation_points(JSONB), snapshot(JSONB)	Thesis 시계열 History
evidence	id, thesis_id, source_type, source_url, vector_doc_id, content_snippet, classification, impact, reason, published_at	신규 정보 근거
analysis_results	id, portfolio_id, thesis_id, analysis_type, bull_summary, bear_summary, judge_summary, concentration_theme, concentration_score, affected_holdings(JSONB), raw_result(JSONB)	Bull/Bear/Judge-Concentration-공통위험 결과
alerts	id, user_id, portfolio_id, thesis_id, severity, title, message, is_sent, sent_at	이메일 알림

Vector Store (pgvector 또는 Qdrant — ADR-0002 참고): [sec_filings](#) / [ir_documents](#) / [earnings_materials](#) / [news_documents](#) 4개 컬렉션, 공통 필드 [id](#), [ticker](#), [doc_meta\(JSONB\)](#), [published_at](#), [chunk_text](#), [embedding](#), [source_url](#).

LangGraph AnalysisState (C 내부 상태, B는 직접 다루지 않음):

```

class AnalysisState(TypedDict):
    portfolio_id: str
    holding_id: str
    ticker: str
    thesis_snapshot: dict
    research_data: dict          # {"filings": [...], "news": [...], "macro": [...]}
    evidence_list: list[EvidenceItem]
    bull_report: str
    bear_report: str
    judge_report: str
    updated_confidence: int      # 0~100
    updated_status: str         # thesis_status 값
    change_reason: str
    conflicting_assumptions: list[str]
    observation_points: list[str]
    alert_decision: dict        # {"severity": "MAJOR", "should_send": true}

```

6. 인터페이스 / API (초안)

엔드포인트	메서드	설명
/api/auth/signup, /login, /me	POST/GET	인증
/api/portfolios	GET/POST	포트폴리오 목록/생성
/api/portfolios/{id}/dashboard	GET	Allocation-Thesis Status-Risk-Theme Dependency 집계
/api/portfolios/{id}/holdings	POST	종목 추가
/api/portfolios/{id}/rebalance	POST	리밸런싱 기록 + 집중도 변화 분석 트리거
/api/holdings/{id}/thesis	POST	자연어 투자 논리 등록 → C 구조화 트리거
/api/theses/{id}, /history	GET/PUT	Thesis 조회/수정/시계열
/api/holdings/{id}/analyze	POST	신규 정보 분석 파이프라인 실행
/api/portfolios/{id}/concentration, /common-risk	GET	집중도/공통위험 결과
/api/portfolios/{id}/query	POST	자연어 질의
/api/alerts, /{id}/read	GET/PATCH	알림 목록/읽음 처리

핵심 함수 계약(B↔C, 함수 직접 호출):

```

def run_analysis_workflow(portfolio_id: str, holding_id: str) -> ThesisAnalysisResult:
    """holding_id 종목의 신규 공시·뉴스·거시데이터를 수집해 기존 Thesis와
    대조하고, 결과를 ThesisAnalysisResult(Pydantic)로 반환한다.
    B는 이 반환값을 그대로 DB에 저장한다."""

```

MCP Tool (B가 서버 구현, C가 LangChain @tool로 래핑):

Tool	함수
SEC MCP	<code>get_filings()</code> · <code>search_filing()</code> · <code>get_company_facts()</code>
Market MCP	<code>get_price()</code> · <code>get_price_history()</code> · <code>get_market_data()</code>
Macro MCP	<code>get_interest_rate()</code> · <code>get_treasury_yield()</code> · <code>get_cpi()</code>
Portfolio MCP	<code>get_portfolio()</code> · <code>update_portfolio()</code> · <code>save_thesis()</code> · <code>get_thesis_history()</code>

7. 기술 스택 & 선택 근거

레이어	선택	주요 대안	근거 요약	ADR
Agent 오케스트레이션	LangGraph	단순 체인, 커스텀 상태머신	조건 분기(Additional Research 루프)·병렬 노드 지원	ADR-0001
Vector DB	pgvector(제안)	Qdrant	기존 PostgreSQL과 통합, 초기 인프라 단순화	ADR-0002
Thesis 판정 방식	Bull/Bear/Judge Agentic Debate	단일 LLM 판정	상반된 근거를 명시적으로 대조해 편향 감소	ADR-0003
Alert 정책	thesis_status 기반 4단계 severity 매핑	고정 규칙 없이 LLM 자유 판단	일관성·예측 가능성 확보	ADR-0004

8. 횡단 관심사

- 보안:** JWT 인증, 사용자별 리소스 접근 제어, `.env`로 비밀값 관리(커밋 금지).
- 성능:** Filing/News/Macro Agent 병렬 수집, LangGraph 노드 간 비동기 처리 고려.
- 관측성:** LangSmith로 Agent 호출·근거·비용 추적, Evidence Classification/Thesis Change Detection Accuracy 등 평가셋 운영.
- 확장성:** MCP Tool 서버와 LangGraph 워크플로를 분리해 향후 데이터 소스(예: 추가 뉴스 API) 확장 시 Filing/News/Macro Agent만 추가.

9. AI 파이프라인 설계

- RAG:** SEC 공시·IR·실적자료·뉴스를 벡터 임베딩해 저장, Evidence Extractor가 관련 청크를 검색해 근거로 사용.
- 프롬프트:** 시스템 프롬프트에 "근거 없으면 판정 금지" 가드레일 명시, Evidence Classification/Thesis Status 출력은 구조화된 JSON 스키마(Pydantic)로 강제.
- 평가:** LangSmith 골든셋으로 Evidence Classification Accuracy·Thesis Change Detection Accuracy·Tool Selection Accuracy·Citation Groundedness·Contradiction Detection Accuracy 측정.

10. 대안 & 트레이드오프

주제	택한 것	포기한 것 / 트레이드오프
Agent 오케스트레이션	LangGraph	단순 체인보다 러닝커브 있지만, Additional Research 루프·병렬 Research Agent에 필수
Thesis 판정	Bull/Bear/Judge 3-Agent	단일 LLM보다 호출 비용·지연 증가, 대신 설명 가능성·편향 감소 확보
Vector DB	pgvector	초대규모 성능은 Qdrant보다 낮을 수 있음(운영 단순함과 맞바꿈) — 팀 논의로 최종 결정 필요

11. 리스크 & 미해결

- LLM이 근거 없이 SUPPORT/CONTRADICT를 판정할 위험 → 가드레일 프롬프트·Additional Research 루프로 완화.
- 외부 API(SEC/시세/거시) 레이트리밋 → MCP Tool 레이어에서 재시도·캐싱 검토 [확인 필요].
- pgvector vs Qdrant 최종 결정 미확정 → ADR-0002에서 팀 합의 필요.
- Additional Research 루프 최대 반복 횟수 미정 → 무한 루프 방지 로직 필요.

12. 배포 & 롤아웃

컨테이너(Docker) 기반 배포. 3인 각자 로컬에서 목데이터/모킹으로 독립 개발 후 dev 브랜치에서 통합, 데모 전 main으로 머지(협업 규칙 1번 참고).

롤아웃: 로컬 통합 테스트 → 팀 내 데모 리허설 → PBL 발표.

13. 관련 ADR

- ADR-0001: Agent 오케스트레이션 — LangGraph 채택
- ADR-0002: Vector DB — pgvector 제안 (Qdrant 대안)
- ADR-0003: Thesis 판정 — Bull/Bear/Judge Agentic Debate 채택
- ADR-0004: Alert 정책 — thesis_status 기반 4단계 severity 매핑 채택